Ders 7 Genişletilmiş Notu: Strategy, State, Template Method ve Visitor Kalıpları

Ders 7 Genişletilmiş Notu: Strategy, State, Template Method ve Visitor Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Strategy Kalıbı: Amaç ve Satır Bölme Örneği
- 2. Strategy Kalıbının Yapısı ve Gerçekleme Ayrıntıları
- 3. Builder ile Strategy Karşılaştırması
- 4. Strategy Uygulama Örneği: Kişisel Finans Programı
- 5. Singleton ile Strategy Birlikte
- 6. State Kalıbı: Amaç, TCP Örneği ve Yapı
- 7. State ile Strategy Arasındaki Farklar
- 8. State Uygulama Örneği: PIM Yazılımında Temiz ve Kirli Kayıtlar
- 9. Template Method Kalıbı
- 10. Template Method Uygulama Örneği: Veritabanı Sorgusu
- 11. Visitor Kalıbı: Amaç ve Derleyici Örneği
- 12. Visitor Kalıbının Yapısı, Etkileşimleri ve Bedeli
- 13. Visitor Uygulama Örneği: Proje Yönetimi Yazılımı
- 14. Kalıpları Öğrenmek Üzerine Soru-Cevaplar
- Örnek Sorular
- Dersin Toparlaması

1. Strategy Kalıbı: Amaç ve Satır Bölme Örneği

Bir önceki derste davranışsal kalıplardan Iterator, Memento ve Observer işlenmişti. Bu ders Strategy kalıbıyla başladı.

1.1 Amaç

Strategy kalıbının esas amacı şudur: aynı işi yapmaya yönelik birden fazla algoritma varsa, bu algoritmaları onları kullanacak istemcilerden soyutlamak. Böylece varılmak istenen nokta, kullanılan algoritmayı çalışma anında değiştirebilmektir.

1.2 Metin düzenleyicilerde alt satıra geçme

Hoca kalıbı bir metin düzenleyicide canlı olarak gösterdi. Bir paragraflık rastgele bir metni Word'de iki yana yasladı. Word bir kelimeyi satıra tam sığdırmak için kelimeler arasındaki boşlukları gerektiği kadar genişletip daraltır. Sayfa genişliği azaltılınca son kelime alt satıra iner ve bu sefer satırdaki boşluklar genişler; genişlik geri alınınca boşluklar yeniden daralır. Bu, alt satıra geçme algoritmalarından yalnız biridir.

Ardından aynı metni WordPad'de gösterdi. WordPad'de üç seçenek vardır:

- Cetvele kaydır: cetveldeki sayfa genişliğine göre alt satıra geçer.

- Pencereye kaydır: pencere ne kadar genişse o kadar alanı kullanır, pencerenin sınırında kelimeyi alt satıra alır.
- Kaydırma yok: hiç alt satıra geçilmez, kullanıcının sağa sola kaydırması gerekir.

Bir başka seçenek de kelimeyi heceden bölmektir. Hoca elle bir bölme işareti koyup genişliği daraltınca kelimenin o noktadan hecelenip alt satıra indiğini gösterdi. Program, boşluklarla oynamak yerine uygun bir hece noktasından kendisi de bölebilirdi.

Sonuç olarak bir metni alt satıra geçirmek için çok sayıda algoritma olabilir ve kullanıcı, WordPad'de olduğu gibi, ayarlarla bu çalışma biçimleri arasında geçiş yapabilmek ister. Strategy kalıbı bu tür işler için önerilmiş bir kalıptır.

1.3 Örnek çözüm

Slayttaki çözüm (ekrandan, 00:13:15) metne şöyle dökülebilir:

```
Uygulama
  - bölmeleyici : SatırBölümleme
  + metinDüzenle()          // not: bölmeleyici.bölmele()
        |
        v  (yönlü ilişki)
«abstract» SatırBölümleme
  + bölmele() {abstract}
        ^ (kalıtım)
        |-- KesmeliBöl    + bölmele()
        |-- KelimedenBöl  + bölmele()
        |-- SınırdanBöl   + bölmele()
```

Uygulama, satır bölme işinden sorumlu bir nesneye (`bölmeleyici`) sahiptir. `metinDüzenle()` metodunun içinde, alt satıra geçmek gereken yerlerde `bölmeleyici.bölmele()` çağrılır. Alt tarafta farklı algoritmaları gerçekleyen sınıflar vardır: kesme karakteriyle bölen, en yakın kelime sonundan bölen, metin ya da pencere sınırından bölen sınıflar. Bunların hepsi soyut `SatırBölümleme` sınıfının alt sınıflarıdır; hoca bunun bir arayüz de olabileceğini söyledi. Üst sınıfta yalnız yapılacak işin imzası vardır.

İleride yeni bir algoritma, tek koşulla, yani `SatırBölümleme` arayüzünü gerçekleyerek eklenebilir. Uygulamaya o an hangi bölmeleyici verilmişse uygulama hemen onunla çalışmaya geçer.

Slayttaki bir ayrıntı: varsayılan bir bölme algoritması varsa üst sınıfın ve ilgili metodun soyut olması zorunlu değildir (ekrandan, 00:13:15). Üst sınıf varsayılan davranışı gerçekler, özel davranışlar alt sınıflarla verilir ve üst sınıfın yerine geçer. Ama kalıbın genel yapısında yapılacak iş soyut bir metottur.

2. Strategy Kalıbının Yapısı ve Gerçekleme Ayrıntıları

2.1 Kalıp yapısı

GoF şeması (ekrandan, 00:16:10):

```
Context                       --strategy-->   Strategy (soyut)
  ContextInterface()          (içerme)          AlgorithmInterface() (soyut)
                                                      ^
                                ConcreteStrategyA / ConcreteStrategyB / ConcreteStrategyC
                                  AlgorithmInterface()
```

Kalıp bileşenleri:

- `Strategy`: desteklenecek tüm algoritmaların ortak arayüzünü belirler.
- `ConcreteStrategyX`: algoritma gerçeklemeleridir.
- `Context`: algoritmaya ihtiyaç duyan nesnedir. Bir ya da birkaç `ConcreteStrategy` nesnesiyle ilişkilendirilir. `ContextInterface()`, gerekli hallerde algoritma gerçeklemesine durum bilgisini açan metot ya da metotlardır.

Hoca GoF şemasındaki gösterim hakkında bir hatırlatma yaptı. Dörtlü Çete'nin kitabı yazıldığında UML henüz standart hâline gelmemişti; bu yüzden kitapta kalıtım okları bugünkü gösterimden farklı görünebilir. Kitapta soyut sınıflar ve soyut metotlar italik yazılır. Bu doğrudur, ama elle çizerken italik belli olmaz. Bu yüzden hoca soyut sınıfı sınıf adının üstüne «`abstract`» stereotipiyle, soyut metodu da imzasının sonuna `{abstract}` yazarak göstermeyi istedi. Kendi şemalarında üye alanları her zaman - (private), metotları çoğunlukla + (public) ile gösterdiğini söyledi.

2.2 Context ile Strategy ilişkisi

Strategy'de genelde context'in bir strateji üyesi olur ve dışarıdan gelen etkilerle context'e "artık şu stratejiyle çalışacaksın" denir. Context'in içinde birden fazla strateji bulunsa bile, hangisine geçileceği WordPad örneğinde olduğu gibi kullanıcı girdisiyle tetiklenir. Context'te farklı algoritmalarla yapılması gereken iş ya da işleri simgeleyen metotlar bulunur. Strateji arayüzünde ise algoritmanın bütün adımlarını simgeleyen soyut metotlar yer alır. Somut stratejiler bunları gerçekler ve context'e hangi somut strateji nesnesi verilirse context o şekilde çalışmaya başlar.

Slayttaki gerçekleme ayrıntıları (ekrandan, 00:20:00):

- `Strategy` ve `Context` nesneleri seçilen algoritmayı gerçeklemek için etkileşimlerde bulunur.
- `Context` nesnesini kullanacak istekçi nesneler, `Context` nesnesine bir `ConcreteStrategy` nesnesi sağlar.

Birinci madde, şemadaki ilişkinin aslında tek yönlü olmadığını anlatır. Strateji kendisini yürütebilmek için context'le ilgili bazı bilgilere erişmek zorunda kalabilir. İhtiyaç öyleyse ilişki tek yönlü de olabilir, ama genelde strateji işini yapmak için context'ten bilgi alır; bu yüzden ilişki çoğu zaman çift yönlüdür. Farklı stratejilerin context'ten isteyeceği bilgiler de farklı olabilir. Bu nedenle context'te, herhangi bir stratejinin ihtiyaç duyabileceği verilere erişimi sağlayan getter türü metotlar bulunmalıdır. Hocaya göre context'le çalışan başka varlıklar da bu metotlara ihtiyaç duyacağı için, büyük olasılıkla yalnız Strategy uygulanacak diye ek metot eklemek gerekmez; gerekiyorsa da eklenir.

2.3 Kullanım yerleri ve zayıf yönler

Kalıbın uygulanabileceği anlar (ekrandan, 00:20:00):

- Bir algoritmanın değişik çeşitlemelerine ihtiyaç duyulduğunda.
- İstekçilerin kullanılacak algoritmanın gerçeklenmesini bilmemesi gereken hallerde.

Hoca kalıbın epey esnek olduğunu ve çalışma zamanına epey dinamizm getirdiğini söyledi. Zayıf yönleri ise şunlardır:

- İstekçiler değişik stratejilerin varlığından haberdar olmak zorundadır. Context kendilerinden bir strateji nesnesi beklediği için istekçi "bu iş için hangi algoritma uygun" diye düşünüp ilgili gerçeklemeyi vermelidir. Varsayılan davranış yeterince iyiyse stratejileri bilmeyen istekçiler de yazılabilir; ama bazı istekçilerin buna ihtiyacı vardır, yoksa bu altyapıyı kurmaya değmezdi.
- Context'in açtığı durum bilgisinin tümüne her strateji ihtiyaç duymayabilir. Context ile strateji arasındaki veri aktarımı push modeliyle yapılıyorsa bu, haberleşmede ek yük getirir. Öyle bir durum yoksa büyük bir sorun değildir.

3. Builder ile Strategy Karşılaştırması

Hoca burada önemli bir tartışma açtı. Epey kalıp görüldü ve yalnız sınıf şemalarına bakınca kalıpların birbirine çok benzediği görülüyor. Slaytta üstte Dörtlü Çete kitabından alınmış Builder şeması, altta Strategy örneğinin şeması vardı. Ana yapı olarak ikisi aynıdır:

- `RTFReader` yerine `Uygulama` gelmiştir.
- `TextConverter` yerine `SatırBölümleme` gelmiştir.
- `ASCIIConverter`, `TeXConverter`, `TextWidgetConverter` yerine `KesmeliBöl`, `KelimedenBöl`, `SınırdanBöl` gelmiştir.

- İki kalıpta da kardeş alt sınıflar çok rahat eklenebilir.

Sınıf şeması açısından tasarımın ana hatları aynıdır, ama önemli farklar vardır. Hangi problemde hangi kalıbın kullanılacağına doğru karar vermek, ilk bakışta benzeyen kalıpların ayrıntılarını ve birbirinden farklı yönlerini iyi bilmekten geçer.

Builder'da karmaşık bir varlık vardır ve parçalarının her biri farklı biçimde oluşturulur. Metinler belli bir biçime çevrilir, ama bazı metin türlerinde bazı bileşen türlerinin anlamı olmaz. Yönetici, her adımı tek tek kendisi denetler ve işin alt adımlarıyla birlikte yapılmasını sağlar.

Strategy'de ise istemci (örnekteki uygulama) strateji gerçeklemesine "bu işi yap" diyerek bütün işi verir. Strateji gerçeklemesi, işin nasıl yapılması gerekiyorsa gerekli bilgileri uygulamadan sorgulayarak alır. Burada denetim akışının çoğunu strateji gerçeklemesi yürütür; Builder'da ise bunu `RTFReader` yapar.

> **Hocanın vurgusu:** Bazı tasarım kalıplarının sınıf şemaları birbirine benzer. Onları farklılaştıran şey amaçları, kullanım yerleri ve gerçekleme ayrıntılarıdır. Builder'da parçaların nasıl oluşturulup birleştirileceği bilinir ve adım adım yaptırılır; Strategy'de istemcinin işin arka planda nasıl yürüdüğünü bilmesine gerek yoktur.

4. Strategy Uygulama Örneği: Kişisel Finans Programı

4.1 Problem

Slayttaki problem (ekrandan, 00:30:00; kaynak DP Explained):

- Uluslararası pazara hitap eden bir kişisel finans programı yazılıyor.
- Değişik ülkelerdeki değişik vergi kurallarıyla işlem yapılması gerekiyor.
- Ana program: `TaskController`.

4.2 Çözüm 1: switch-case

İlk akla gelen çözüm switch-case'tir. Ülkeyi ya da bölgeyi simgeleyen bir `enum` olur; istenirse başka ülkeler de eklenir. `TaskController`'da bir ülke kodu alanı, onun için bir setter ve gelire göre vergiyi döndüren bir metot olur. Slayttaki kod (ekrandan, 00:30:00):

```
package strategy.solution1;
public enum Country { TR, US, EU }
public class TaskController {
    private Country code;
    public void setCountry(Country code) {
        this.code = code;
    }
    public double taxAmount(double income) {
        switch (code) {
            case TR: return income * 0.18;
            case EU: return income * 0.15;
            case US: return income * 0.12;
            default: return income * 0.10;
        }
    }
}
```

Varsayılan durum konmayabilir de; burada konmuş ve %10 vergi kesilmiştir. Şemada `TaskController(code : Country, + taxAmount(income : double) : double, + setCountry(code : Country))` «Enum» `Country`'ye (`TR, US, EU`) kesikli bağımlılık okuyla bağlıdır.

Bu kadar basit bir kodda switch-case sorun gibi görünmeyebilir. Ama gerçekte her durumun içi o kadar ayrıntılı ve karmaşık olur ki `taxAmount()` devasa ve anlaşılması çok zor bir metoda dönüşür. Yeni bir ülke ya da bölge eklenince yeni bir enum değeri ve yeni bir durum eklenir; metot her seferinde daha da büyür.

> **Hocanın vurgusu:** Kendinizi switch-case kodlarken görüyorsanız, bu büyük olasılıkla orada kalıtım ve çok biçimliliğe dayanan daha iyi, daha doğru bir tasarım yapmanız gerektiğinin işaretidir.

4.3 Çözüm 2: Strategy

İkinci çözümde uygulamada `TaxRule` türünden bir üye alan olur; strateji ilişkisini bu alan kurar. Her bölgenin kodu yalnız kendi sınıfında yer alır. Slayttaki kod (ekrandan, 00:33:30):

```
package strategy.solution2;
public interface TaxRule {
    public double taxAmount(double income);
}
public class TaskController {
    private TaxRule tax;
    public void setTax(TaxRule tax) { this.tax = tax; }
    public double taxAmount(double income) {
        return tax.taxAmount(income);
    }
}
public class TRrules implements TaxRule {
    public double taxAmount(double income) {
        return income * 0.18;
    }
}
```

`TaskController`'ın vergi hesaplama metodu yalnız işi strateji gerçeklemesine iletir: "sen bunu hesapla ve bana döndür, ben de aynen geri döndüreyim." `TRrules` arayüzü gerçeklediğini beyan eder ve metodun gövdesini kodlar.

Gerçekte her bölgenin hesabı tek satır olmayacaktır. Ama aynı karmaşıklıkta üç ayrı `taxAmount()` metodu yazmak bir şeydir, hepsini tek noktada toplamak başka bir şey. Strategy ile hata düzeltmek ya da bir gereksinim değişikliğini kodlamak için yalnız ilgili sınıfın metoduna odaklanmak yeter. "Kontrol etmesi kolay, ararız" denebilir; ama kişisel finans uygulamasında yalnız vergilendirme olmayacak, farklı ülkelerde başka işlevler de farklı çalışacak. Switch-case'te `case TR` parçalarını programın her yerinde aramak gerekir. Strategy'de bir ülkeyle ilgili işin bir strateji gerçeklemesinde olduğu bilinir ve kod rahatça değiştirilir. Ayrıca çalışma anında "bu ülkedeydim, şu ülkeye geçtim" gibi değişiklikler de rahatça yapılır.

4.4 Başka mali işlemler

Hoca vergi dışında başka mali işlemlerin ne olabileceğini sordu ve kendisi yanıtladı: yatırım işlemleri (her ülkede yatırım hesabı açmak ve işlem yapmak farklı çaba ister), döviz alım satımı (bazı ülkelerde sabit komisyon vardır, bazılarında yoktur), zamanla icat edilen yeni ürünler (hoca örnek olarak Türkiye'deki kur korumalı mevduatı verdi), emeklilik işlemleri.

Böyle bir durumda genel tavsiye, ne kadar farklı mali iş varsa o kadar Strategy kalıbı kurmaktır: yatırım için bir arayüz ve altında `TRInvestment`, `EUInvestment`, `USInvestment` gibi sınıflar; emeklilik için ayrı bir kalıp. Bir varyasyon olarak bütün işlerin metotları tek bir strateji arayüzünde tanımlanabilir; böylece bir ülkeyle ilgili her şey tek sınıfta durur. Bu derli toplu olması açısından güzeldir, ama nesneye yönelimin genel tavsiyesi "parçala, böl, yönet"tir. Bu yüzden ayrı işlemler için ayrı Strategy kalıbı kurmak önerilir; yüzde yüz zorunlu değildir.

5. Singleton ile Strategy Birlikte

Singleton ile Strategy birlikte kullanılabilir. Strateji gerçeklemeleri genelde Singleton olmaya adaydır. `TRrules` örneğinde Türkiye'de vergi kuralları herkes için aynıdır; kanun farklı kişilere farklı yükümlülükler koysa da o ülkede kanun tektir. Aynısı Avrupa ve ABD için de geçerlidir.

Singleton'da o türden tek bir nesne olabilir. O tek üye `static` tanımlanır, kurucu dışarıya açılmaz ve `getInstance()` metodu ortak paylaşılan nesneyi döndürür. Slayttaki kod (ekrandan, 00:40:00):

```java
package singleton.example;
public class TRrules implements TaxRule {
    private static TRrules instance;
    private TRrules() {
        //gerekli ilklendirmeler
    }
    public static TRrules getInstance() {
        if (instance == null)
            instance = new TRrules();
        return instance;
    }
    public double taxAmount(double income) {
        return income * 0.18;
    }
}
```

`EUrules` ve `USrules` sınıfları da benzer şekilde gerçeklenir.

6. State Kalıbı: Amaç, TCP Örneği ve Yapı

Hoca State'i Strategy'den hemen sonra işlemenin, iki kalıbın benzerlik ve farklarını görmek için iyi bir fırsat olduğunu söyledi.

6.1 Amaç

State kalıbının amacı, bir nesnenin iç durumu değiştiğinde davranışının da değişmesini sağlamaktır. Şu itiraz gelebilir: nesnenin durumu zaten üye alanlarının o anki değerlerinin birleşimidir ve üye alanlar değiştikçe nesnenin hesaplamaları da değişir. Bu doğrudur, ama burada basit bir parametrik değişiklikten söz edilmiyor. Küçük ölçekli değişim, bir üye alanın değeri yüzünden bir metodun sonucunun değişmesidir. Bu kalıpta amaçlanan ise nesnenin önceki çalışma biçiminin tamamen değişmesidir.

6.2 TCP bağlantısı örneği

Bir haberleşme kütüphanesinde TCP gerçeklemesi yapılıyor. TCP bağlantısı herhangi bir anda üç durumdan birindedir: `Established`, `Listen` ve `Closed`. Bağlantı o an hangi durumdaysa ona göre çok farklı biçimde çalışır.

Burada da switch-case akla gelebilir, ama onun sakıncaları görüldü. Temel ilke "neyin değiştiğini bul ve onu sarmala"dır. Burada değişen TCP durumlarıdır. O hâlde bir TCP durumu sınıfı olur ve bu sınıf soyut olmak zorundadır, çünkü genel geçer bir "durum" yoktur; bağlantı üç durumdan birinde olmak zorundadır, bu yüzden genel durum sınıfından doğrudan nesne oluşturulamamalıdır. Bağlantı sınıfında bu arayüz ya da soyut üst sınıf türünden bir `state` üyesi olur. `open`, `close`, `acknowledge` gibi işler o anki somut durum nesnesine iletilerek yürütülür.

Ek olarak durumlar arasında geçiş kuralları vardır. Hocanın anlatımıyla bir TCP bağlantısı açıldığında önce kurulu (established) duruma, bir onay (acknowledge) aldıktan sonra dinleme durumuna geçer; kapatma komutu gelene kadar bilgi aktarılır; dinleme durumundan kapalı duruma geçilebilir, ama belki tekrar kurulu duruma geçilemez. Yani "şu durumdayken şu koşullar gerçekleşirse şu duruma geçilir" türünden kurallar vardır.

> **Ders sonrası not:** Hoca bağlantının önce kurulu duruma, sonra dinleme durumuna geçtiğini anlattı. Gerçek TCP'de sıra büyük olasılıkla tersidir: sunucu tarafı kapalı durumdan dinleme (LISTEN) durumuna geçer, üçlü el sıkışma tamamlanınca kurulu (ESTABLISHED) duruma geçer. Örnek kalıbı anlatmak içindir; geçiş sırası kalıbın yapısını değiştirmez.
>
> **Hocanın vurgusu:** State kalıbını Strategy'den ayıran en önemli nokta durumlar arasındaki geçiş kurallarıdır.

6.3 Kalıp yapısı

GoF şeması (ekrandan, 00:50:00):

```
Context                          --state-->   State (soyut)
  Request()                      (içerme)       Handle() (soyut)
  // not: state->Handle()                          ^
                                        ConcreteStateA / ConcreteStateB
                                          Handle()
```

Kalıp bileşenleri:

- `Context`: davranışı durum bilgisiyle birlikte değişen varlığı simgeler.
- `State` / `ConcreteStateX`: değişen davranışın arayüzü ve gerçeklemeleridir.

Gerçekleme ayrıntısı olarak slaytta "durum değişiminden genellikle Context sorumlu olur; aksi halde?" yazar (ekrandan, 00:50:00). State'te context ile durumlar arasında gerçekten bire-çok ilişki olabilir, çünkü context'in içinde bulunabileceği birden fazla durum vardır ve bu durumların her birini context'in kendisi bilir; durum dışarıdan verilmez. Context, geçirdiği etkileşimler nedeniyle bir durumdan başka bir duruma geçer. Strategy'de ise strateji dışarıdan verilir ve verilen strateji gereğini yapar. Kaç durum varsa her biri için bir somut durum sınıfı kodlanır.

Kalıbın kullanılabileceği yerler:

- Bir nesnenin çalışma biçimi çalışma anında o nesnenin durumuna göre değişiyorsa.
- Bir nesnenin durumuna bağlı, karmaşık, uzun ve çok seçenekli karar komutları gerekiyorsa.

6.4 Soru-cevap: TCP örneği ve Chain of Responsibility

Soru: TCP örneğinde aklıma Chain of Responsibility geldi; bir nesne var, farklı durumları var ve aralarında bir geçiş, bir zincir var. Sınavda ya da gerçek hayatta böyle bir durumla karşılaşınca hangi kalıbı kullanacağımı, aralarındaki farkı nasıl bulabilirim?

Cevap: Bunun kolay bir yanıtı yok. Kalıpları, ayrıntılarını ve çözdükleri problemleri tam bilmek gerekir; önümüze çıkan yeni problem hangi kalıbın çözdüğü probleme en çok benziyorsa ona göre karar verilir. Chain of Responsibility'de iş, onu en iyi yapacak nesneye yönlendirilir. TCP örneğinde ise tek bir iş vardır, bağlantının kurulup sürdürülmesi, ama bunun farklı aşamaları vardır. Bu aşamaları gerçeklemek için en uygun kalıp State'tir. Builder gibi parçaları bir araya getirmek ya da Decorator gibi çalışma anında farklı sorumluluklar eklemek gerekmiyor. İşin nasıl yapılacağı bütün ayrıntıları ve kurallarıyla bellidir: hangi aşamada ne olursa hangi çalışma biçimine geçileceği. Bunun kısa yolu yoktur; kalıpları hatırlayarak, kullanarak, tekrarlayarak elde edilen bir deneyimdir.

Hoca ayrıca, bir soru için hiç düşünmediği bir kalıbı öğrencilerin gerekçesiyle savunup gereksinimleri karşıladığını gösterdiği durumlarda o yanıtı da geçerli yanıtlar arasına eklediğini, bunun karşılıklı bir öğrenme olduğunu söyledi.

7. State ile Strategy Arasındaki Farklar

Hoca iki kalıbın farklarını şöyle sıraladı:

- **Kim seçer:** State'te context'in durumu değiştikçe context kendi somut durumunu kendisi seçer ve ona göre hareket eder. Strategy'de hangi stratejinin kullanılacağı dışarıdan verilir; context kendisine gelen parametreye göre işlem yapar.
- **Geçişlerin açıklığı:** State'te durum geçişleri açıkça kodlanır. Somut durum gerçeklemeleri arasında da ilişki olabilir; sahiplik değil ama kesikli çizgili bağımlılık okuyla birbirlerini görebilirler. Hiç olmazsa bir durum, kendisindeki geçiş kuralı tetiklendiğinde context'e "benim için bu kadar, şimdi şu duruma geç" demek için context'in metodunu çağırabilir. Kısacası geçişlerin kodu context ya da durum gerçeklemelerinde açıkça bulunur. Strategy'de bu bilgi daha az açıktır, strateji genelde dışarıdan verilir.
- **Geçiş sıklığı:** State'te durum geçişleri nispeten sıktır, özellikle dinamik bir yapıda. Strategy'de strateji değişimi seyrektir: bir strateji verilir ve dışarıdan bir istek gelip "artık şu stratejiyle çalışacaksın" diyene kadar aynı strateji kullanılır.

8. State Uygulama Örneği: PIM Yazılımında Temiz ve Kirli Kayıtlar

8.1 Problem ve sınıf şeması

Bir kişisel bilgi yöneticisi (Personal Information Manager, PIM) hazırlanıyor: kişiler, erişim bilgileri, yapılacak iş listeleri gibi kayıtlar tutuluyor. Kayıtlar düzenlendikçe iki durumdan birinde olur: temiz (clean) ya da kirli (dirty). Kirli, kayıtta değişiklik yapılmış ama henüz kaydedilmemiş demektir; değişiklikler kaydedilince kayıt temiz duruma geçer. Hoca çözümü Applied Java Patterns kitabından aldığını söyledi.

Sınıf şeması (ekrandan, 00:54:00):

```
State (soyut)                          PIMApplication
  + edit()                <--------      + CLEAN : State      (static)
  + save(f: File, s: Serializable)       + DIRTY : State      (static)
     ^                                   - currentState : State (static)
     |-- CleanState                      + getCurrentState() : State
     |     + edit()                      + setCurrentState(state: State)
     |     + save(f: File, s: Serializable)
     |-- DirtyState
(PIMApplication, CleanState ve DirtyState'e de oklarla bağlı)
```

`PIMApplication`'da iki durum `public static final` olarak verilmiştir. Bunlar tek örnekli ve değişmeyecek olduğu için `public` de yapılabilir. Şu anki durumu tutan `currentState` ise `private`'tır ve uygulama ilk açıldığında durum temizdir. `edit` isteği geldiğinde mevcut durumun `edit()` metodu, `save` isteği geldiğinde mevcut durumun `save()` metodu çalışır.

Durum diyagramı olarak: temiz durumda `edit` olursa kirli duruma geçilir. Kirli durumda `save` dışında ne yapılırsa yapılsın kirli kalınır. `save` yapılınca kirli durumdan temize geçilir.

8.2 Ortak işlerin üst sınıfta toplanması

Hoca, `save` işlemi temiz ve kirli durumda aynı biçimde yapılacağı için soyut `State` sınıfında somut bir metot olarak kodlanabileceğini söyledi. Bu kalıpta söylenmez, ama yapılabilir: bütün durumlarda ortak yürütülen bir iş varsa üst soyut sınıfa yazılır. Bütün durumlarda ortak bir nokta bulmak genelde mümkün olmaz. Olası on durumdan sekizinde ortak bir iş varsa yine ortak kısım üst sınıfta kodlanır, farklı olan iki durumda metot ezilir. Örnekte `State` sınıfında hiçbir metot soyut bırakılmamış, yalnız sınıf soyut yapılmıştır. Üst sınıftaki `save()` kaydetme işini yapar ama durum geçişiyle ilgili bir şey içermez; içermemelidir de, çünkü geçişler somut durumlarda çok farklı biçimlerde yürütülür.

8.3 Slayttaki ilk kod ve hata

Slayttaki ilk kod (ekrandan, 01:00:00):

```
package dp.state.example;
import java.io.*;
public class CleanState extends State {
    public void edit() {
        StateFactory.setCurrentState(PIMApplication.DIRTY);
        super.edit();
    }
    public void save(File f, Serializable s) throws IOException {
        super.save(f, s);
        StateFactory.setCurrentState(PIMApplication.CLEAN);
    }
}
```

```
package dp.state.example;
public class DirtyState extends State { } //marker class
```

`CleanState`'te `edit` gelince durum kirli yapılır, düzenleme işi üst sınıfa iletilir. `save` gelince kaydetme başarılıysa durum temiz yapılır. `DirtyState` ise hiçbir metodu olmayan bir işaretleyici

sınıftır (marker class). Hoca bunu Java'daki `Cloneable` ve `Serializable` arayüzlerine benzetti: bir sınıfın nesnelerinin kopyalanabilmesi için yalnız `implements Cloneable` denir; arayüzde hiç metot yoktur, yalnız bir işaret verilir.

Hoca kodu incelerken bir sorun fark etti. `DirtyState` boş olduğu için `save()` metodunu kalıtımla soyut `State`'ten alır ve o metotta durum geçişi yoktur. Yani kirli bir kayıt kaydedildiğinde temiz duruma dönmez. Bunun iki çözümü vardı: geçişli `save()`'i üst sınıfa taşımak ya da kirli durumda da geçişi kodlamak.

> **Hocanın vurgusu:** Bir üst sınıf alt sınıfları hakkında bilgi sahibi olmamalıdır. Soyut `State` sınıfı hangi somut durumların olacağını bilmemelidir; bu yüzden durum geçişini üst sınıfa yazmak istemiyorum. Yapılan işler yüzde yüz ortak değilse alt sınıflarda kodlanması daha mantıklıdır.

8.4 Düzeltme

Hoca `DirtyState`'i işaretleyici sınıf olmaktan çıkarıp `CleanState`'teki kodu kopyaladı ve `edit()` kısmını sildi, çünkü kirli durumda düzenleme yapılınca yine kirli kalınır, yapılacak bir şey yoktur. Düzeltilmiş hâl (ekrandan, 01:10:00):

```
package dp.state.example;
import java.io.*;
public class DirtyState extends State {
    public void save(File f, Serializable s) throws IOException {
        super.save(f, s);
        StateFactory.setCurrentState(PIMApplication.CLEAN);
    }
}
```

Böylece temiz ve kirli durumdaki `save()` metotları tamamen aynı oldu; bir tür yinelenen kod oluştu. Hoca bunun istenirse üst sınıfa taşınabileceğini, ama daha saf bir State örneği olması için durum geçişlerinin ya durumların kendi içinde ya da context'te yönetilmesini tercih ettiğini, bu yüzden kodun böyle kalacağını söyledi. Ayrıca temiz durumdaki `edit()` yalnız `super.edit()` çağırıp geçiş yapsaydı ezilmesine gerek kalmayacağını, burada geçiş olduğu için ezildiğini belirtti.

Hoca bir hata daha düzeltti: kodda `StateFactory.setCurrentState(...)` geçiyordu, ama şemada böyle bir fabrika sınıfı yoktu. Doğrusu, geçerli durumu tutan `PIMApplication`'ın `setCurrentState(...)` metodunu çağırmaktır; hoca slaytları buna göre düzeltti. Durumlar Singleton yapılmak istenirse bir durum fabrikası o bağlamda yararlı olabilir, ama bu örnekte ihtiyaç yoktur.

9. Template Method Kalıbı

9.1 Amaç ve GoF örneği

Template Method kalıbının amacı, bir algoritmanın ya da iş akışının bazı adımlarını belirleyip geri kalanların ayrıntılarının belirlenmesini ertelemek, başka deyişle alt sınıflara bırakmaktır.

Dörtlü Çete'nin örneği şöyledir: farklı belge türleriyle çalışabilecek uygulamaların geliştirilmesini sağlayan bir çerçeve (framework) program hazırlanıyor. Bu çerçevenin kullanıcıları aslında onu kullanarak başka programlar üretecek yazılımcılardır. Yazılımcılar çerçevenin sunduğu belge ve uygulama sınıflarından kalıtım yoluyla kendi sınıflarını üretecek, böylece kendi yazılımlarını hazırlayacaktır. Bir belgeyi açarken ve oluştururken belgenin ve uygulamanın türünden bağımsız yapılabilecek işler vardır. Her türlü inceliği kullanıcının sırtına yüklemek istenmez; ortak temel adımların doğru sırayla yürütülmesini çerçeve düşünmelidir, ancak o zaman böyle bir çerçeve yararlı olur.

Hoca şemada çizdiği bir çizginin üstünü çerçeve, altını onu kullanan uygulama olarak gösterdi. Çerçevede `Application` ve `Document` sınıfları soyuttur; bir uygulama çok sayıda belgeyle çalışır (parça-bütün, bire-çok). Bir belgenin nasıl okunacağı belgenin ayrıntılarına göre çok farklıdır; bunu çerçevede kodlamak mümkün değildir, kalıtımla kendi belge sınıfını yazan yazılımcı kod-

lar. Uygulamaya yeni bir belge eklemek, belgeler bir listede tutuluyorsa listeye bir eleman eklemek gibi ortak bir iştir.

`Application` sınıfında üç metot soyut bırakılmıştır, diğerleri somuttur:

- Belge oluşturmak, çünkü farklı uygulama ve belge türlerinde çok farklıdır.
- Uygulamanın belgeyi açabilip açamayacağını denetlemek (`CanOpenDocument()`), çünkü belgenin türüne göre değişir.
- Belge açılmadan önce yapılacak ön hazırlıklar, çünkü uygulama türüne bağlı olabilir.

`OpenDocument()` metodunun akışı şöyledir:

- Önce `CanOpenDocument()` ile belgenin açılıp açılamayacağı denetlenir; açılamıyorsa kullanıcıya bilgi verilir ve işlem yapılmaz.
- Belge, türüne bağlı biçimde oluşturulur (soyut adım).
- Oluşan belgeye bir işaretçi alınır ve belge listesine eklenir; bu her durumda bu sırada yapılır.
- Belge türüne göre değişen ön hazırlıklar alt sınıfta yapılır (soyut adım).
- Belge gerçekten açılır ve okunur.

Somut metodun içinde bazı adımlar somut olarak kodlanmış, bazıları soyut bırakılıp nasıl yapılacaklarının belirlenmesi alt sınıflara bırakılmıştır. Kalıbın çalışma biçimi budur.

9.2 Kalıp yapısı

GoF şeması (ekrandan, 01:36:00):

```
AbstractClass
  TemplateMethod()        // not: ... PrimitiveOperation1() ... PrimitiveOperation2() ...
  PrimitiveOperation1()   (soyut)
  PrimitiveOperation2()   (soyut)
     ^
ConcreteClass
  PrimitiveOperation1()
  PrimitiveOperation2()
```

Kalıp bileşenleri:

- `ConcreteClass`: algoritmanın alt düzey ayrıntılarını gerçekler.
- `AbstractClass`: `TemplateMethod()` içinde algoritmanın üst düzey ya da değişmeyecek ayrıntılarını gerçekler, alt düzey ayrıntıları `ConcreteClass` alt sınıflarına aktarır.

Gerçekleme ayrıntıları (ekrandan, 01:36:00):

- Alt düzey metotlar alt sınıflarda mutlaka tanımlanmalıdır: Java'da `abstract`, C++'ta "pure virtual" olmalıdır.
- Üst düzey metot (`TemplateMethod`) alt sınıflar tarafından yeniden tanımlanamamalıdır, `final` olmalıdır.

Şablon metodun içinde soyut ve somut işler olması gereken sırada birlikte yürütülür. Alt sınıfa düşen tek iş soyut metotları somutlaştırmaktır. Şablon metot `final` yapılınca soyut sınıf işin nasıl yürütüleceğine dair değiştirilemez bir şablon ortaya koyar; alt sınıflar yalnız serbest bırakılan alanda iş yapabilir. Sınıfın kendisi ise `final` yapılmamalıdır, yoksa alt sınıf oluşturulamaz.

9.3 Kullanım yerleri

- Bir algoritmanın ya da iş akışının değişmez kısımlarını bir kez kodlayıp değişebilecek kısımlarını alt sınıflar üzerinden değiştirmeye açık bırakmak istediğimizde.
- Yeniden yapılandırma (refactoring) açısından: kardeş sınıflarda çok tekrarlanan ve bazen hatalara yol açacak biçimde tutarsız kodlanmış kodlar görürsek, bunları mümkün olduğunca az hataya kapı açacak biçimde üst sınıfta toplamak istediğimizde.

Hoca kalıbın alt sınıflar üzerine bazı kurallar dayattığını vurguladı. Normalde üst sınıfın alt sınıflar hakkında bilgisi olmamalıdır. Bu kalıp bu kurala uyar, alt sınıflar hakkında bir şey bilmez;

ama alt sınıfların bazı işleri yapmasını kısıtlar (şablonu değiştiremezler), bazılarını yapmasını ise zorunlu kılar (soyut metotları somutlaştırmalıdırlar).

10. Template Method Uygulama Örneği: Veritabanı Sorgusu

10.1 Problem

Slayttaki problem (ekrandan, 01:40:00; kaynak DP Explained):

- Bir programda veritabanından kayıtlar çekilecek.
- Bu işlem SQL ile bir `SELECT` komutu hazırlanarak verilebilir.
- Programın farklı VTYS'lerle çalışabilmesi gerekmektedir.
- Farklı VTYS'lerde VT ile bağlantı kurma ve SQL komutlarını biçimlendirme farklı olabilir. Özellikle saklı yordam (SP) dilleri VTYS'ler arasında çok farklılık gösterir.
- Tüm farklılıklara rağmen kayıt çekme işleminin değişmeyen temel adımları vardır:
  - Bir bağlantı komutu biçimlendirilir (CONNECT).
  - VT'ye bağlantı komutu gönderilir.
  - Bir kayıt seçme komutu biçimlendirilir (SELECT).
  - VT'ye seçme komutu gönderilir.
  - Seçilen veri kümesi geri döner.
- Ortak temel adımların olması, Strategy yerine Template Method kalıbının kullanımını daha doğru bir seçim kılar.

Hoca son maddeyi açtı: farklı VTYS'leri desteklemek için ilk akla Strategy gelebilir, "veritabanıyla ilgili bir iş var, al bunu yap" diye işi bir strateji gerçeklemesine devretmek cazip görünür. Ama burada ortak noktalar vardır. State'te ve daha çok Strategy'de böyle ortak noktalar yok denecek kadar azdır; Template Method'da ise ortak yapılacak işler vardır. Bu yüzden burada Template Method seçilmelidir.

10.2 Çözüm

Slayttaki kod (ekrandan, 01:45:15):

```
package templateMethod.example;
public abstract class QueryTemplate {
    public final void doQuery(String DBname,
                              String querySpec) {
        String dbCommand;
        dbCommand = formatConnect(DBname);
        //dbCommand komutunu yürüterek bağlantıyı kur.
        dbCommand = formatSelect(querySpec);
        //dbCommand komutunu yürüterek kayıtları oku.
        //Oluşan veri kümesini geri döndür.
    }
    abstract String formatConnect(String DBname);
    abstract String formatSelect(String querySpec);
}
```

`doQuery()` şablon metodudur ve `final`'dır. Hoca bunun nedenini şöyle açıkladı: işin bütün espirisi şablonun o şekilde uygulanmasının zorunlu olmasıdır. `final` denmezse alt sınıflar metodu ezip işi kafalarına göre yürütebilir; bu da yapmak istediğimiz şeyin tam tersidir.

Şablonun içinde önce veritabanı adına göre bağlantı komutu biçimlendirilir ve yürütülür; ardından sorgu tanımına göre seçme komutu biçimlendirilir, yürütülür, kayıtlar okunur ve oluşan veri kümesi döndürülür. VTYS'ye göre farklı yapılacak olan `formatConnect()` ve `formatSelect()` soyut metotlardır. Bu soyut metotlar paket görünürlüğünde tanımlanmıştır; hoca bunun her zaman böyle olmasının şart olmadığını, kalıbın bu yönde bir zorlama yapmadığını söyledi.

Somut sınıflar çalışılan VTYS'nin ihtiyaçlarına göre bağlantı ve seçme komutlarını oluşturup döndürür: Oracle için bir `OracleQueryTemplate`, SQL Server için bir `SQLServerQueryTemplate`. Hoca graf veritabanları gibi ilişkisel olmayan sistemlerde işlerin çok daha farklı yürüyeceğini, bunun şablonun gerekliliğini daha da iyi gösterdiğini ekledi. Ana uygulamada `QueryTemplate`

türünden bir üye alan olur ve kurucuda verilir; hangi veritabanıyla çalışılacaksa onun somut gerçeklemesi verilince işler o soyut bağlantı üzerinden yürür.

> **Hocanın vurgusu:** Template Method'un Strategy'den farkı, işin komple farklı yapılması değil, işin bazı adımlarının mutlaka o şekilde ve o sırada yapılmasını zorlayıp yalnız bazı alt adımları serbest bırakan bir şablon vermesidir.

11. Visitor Kalıbı: Amaç ve Derleyici Örneği

11.1 Amaç

Elimizde zincir ya da ağaç biçiminde bir nesne yapısı, yani bir koleksiyon ya da parça-bütün yapısı var. Bu yapıyı oluşturan elemanlar üzerinde bazı işlemler tanımlamak istiyoruz. Yapı tek ya da çift yönlü bağlantılı olabilir. Bu işlemleri elemanların sınıflarına eklemek istemiyoruz; elemanların sınıf yapısını değiştirmek de istemiyoruz. Bunun nedenleri şunlar olabilir:

- Kullanacağımız varlıkların kaynak kodu elimizde ya da bize açık değildir.
- Geriye doğru uyumluluğu korumak için sınıf yapılarını değiştirmek istemiyoruz.
- Altyapıyı ilk kez kuruyoruz ve uzun süre sahada kalacağını öngörüyoruz; varlıkları sık sık değiştirmek yerine altyapıyı bir kez hazırlayıp sonraki istekleri onun üzerinde daha rahat gerçeklemek istiyoruz.

Hoca bu duruma bir eleştiri getirilebileceğini kabul etti: madem yapılacak işlemler ve sorumluluklar tam belirlenmemiş, sınıf yapısını değiştirirsek belki daha doğru, daha esnek bir tasarıma varırız. Bu haklı bir eleştiridir. Ama pratik zorunluluklar nedeniyle bu karar veriliyorsa, "madem bu yola girdik, bari doğru ilerleyelim" anlayışıyla bu kalıp kullanılır.

11.2 Derleyici örneği

Dörtlü Çete'nin örneği bir derleyicidir. Kaynak kod farklı türden token'lara ayrılır ve her token ayrı ele alınır. Token'lar bir ağaç yapısında düğümler oluşturur. Derleyici bu ağacın üzerinden tek seferde geçmek zorunda değildir; birden fazla kez geçebilir ve her geçişte farklı parçalar üzerinde farklı işlemler yürütür.

Hoca tahtaya `x = 3 + 5` ifadesini yazdı. Derleyici bunun için bir ayrıştırma ağacı (parse tree) oluşturur: bir atama işlemi vardır; atamanın hedefi `x` değişkenidir; atanan değer bir toplama işleminin sonucudur; toplamanın bir ucunda 3, diğer ucunda 5 sayısı vardır. Ardından derleyici ağacı dolaşarak tip denetimi yapar: `x`'in türü ile toplama işleminin oluşturacağı tür uyumlu mu? Derleyici bir IDE'ye gömülüyse ağacı ikinci kez dolaşıp renklendirme yapar, örneğin üye alanları mavi, geçici değişkenleri gri gösterir.

Benzer bir token farklı bağlamda farklı anlam taşıyabilir. Hoca örnek olarak Java'nın `+` işlecini verdi: işlecin iki yanında string varsa yine benzer bir ayrıştırma ağacı oluşur, ama bu sefer işlem aritmetik toplama değil string birleştirme (concatenation) olarak çalışır ve üretilecek kod bambaşka olur. Bu, `+` işlecinin aşırı yüklenmesinin (overloading) örneğidir.

İlk akla gelen çözümde düğümlerde tip denetimi, kod üretimi, güzel yazdırma (pretty printing) gibi işlemler olur. Değişken referansı düğümü, atama düğümü gibi kardeş düğüm sınıfları vardır ve her işlemin parçaları farklı düğüm türlerine dağılır. Hoca bunu "tam bir bakım kabusu" diye nitelendirdi: tip denetimi kurallarında bir değişiklik olunca ilgili olabilecek bütün alt sınıflarda tip denetimi kodu ele alınıp düzeltilmelidir. Ayrıca her düğümün her işleme ihtiyacı olmayabilir; örneğin bir düğümde kod üretimi ve tip denetimi gerekmezken yalnız renkli ya da italik yazdırma gerekebilir. Bu durumda bazı sınıflarda boş gövdeli metotlar oluşur.

12. Visitor Kalıbının Yapısı, Etkileşimleri ve Bedeli

12.1 Örnek çözüm

Kalıbın önerisi: derleyici parçaları tek tek ziyaret etsin ve her parça üzerinde yalnız gerekli işlemleri yapsın. Kalıp adını da bu ziyaret edici sınıftan alır. Slayttaki çözüm (ekrandan, 02:00:00):

```
NodeVisitor (soyut)
  VisitAssignment(AssignmentNode)
  VisitVariableRef(VariableRefNode)
     ^
     |-- TypeCheckingVisitor      VisitAssignment(...), VisitVariableRef(...)
     |-- CodeGeneratingVisitor    VisitAssignment(...), VisitVariableRef(...)

Program ◇--> Node (soyut)
               Accept(NodeVisitor)
                  ^
                  |-- AssignmentNode    Accept(NodeVisitor v)   // v->VisitAssignment(this)
                  |-- VariableRefNode   Accept(NodeVisitor v)   // v->VisitVariableRef(this)
```

Soyut `NodeVisitor` sınıfında, dolaşılacak her düğüm türü için bir `visit` metodu vardır ve hepsi soyuttur. Tip denetimi için dolaşan ziyaretçi atama düğümünde başka, değişken referansı düğümünde başka işler yapar; kod üreten ziyaretçi de bu düğümlerde farklı işler yapar.

Bu tek başına yeterli değildir. Düğümlerin her biri bir ziyaretçiyi kabul edebilmek için `Accept` metoduna sahip olmalıdır. Bileşenler arasında zaten bir hiyerarşi olacağı için yapılacak tek şey, hiyerarşinin üst sınıfına soyut bir `Accept` metodu eklemek ve düğümlerde bu metodun içini kodlamaktır. `Accept`'in yaptığı iş şudur: örneğin atama düğümü kendisini ziyaret etmek isteyen ziyaretçiyi kabul eder, ama "ben bir atama düğümüyüm, beni atama düğümü olarak ziyaret edeceksin" diyerek ziyaretçinin `VisitAssignment` metodunu kendisini parametre vererek çağırır; kendini tamamen açmaz. Değişken düğümü de "ben bir değişken düğümüyüm, beni bu şekilde ziyaret et" der. Böylece her parçada yalnız gerekli işlemler yürütülür.

Slayttaki maddeler (ekrandan, 02:00:00):

- Düğüm gerçeklemeleri, kendi üzerlerindeki gerekli işlemleri çağıracak ziyaretçi gerçeklemelerini kabul eder (`Accept` metodu ile).
- Bu şekilde parçalarda yalnız gerekli işlemler yürütülür.
- Çözümün zayıf yönü: düğümler ziyaretçinin hangi metodunun bu düğüm türü için olduğunu bilmelidir.

Zayıf yönden dolayı düğümlerden ziyaretçiye ister istemez ek bağımlılıklar kurulur. Hoca bu bağımlılığın mümkünse soyut ziyaretçi üst sınıfına doğru olması gerektiğini söyledi. Bazı durumlarda bir düğümün belirli bir somut ziyaretçi tarafından ziyaret edileceğini bilmesi gerekebilir, ama bu olabildiğince olmamalıdır; kalıbın istediği, düğümün soyut sınıfta tanımlı `visit` metotlarından kendisine uygun olanı çağırmasıdır.

12.2 Kalıp yapısı

Kalıp yapısında soyut bir `Visitor` ve her somut eleman türünün ziyaret edilmesini sağlayacak soyut metotlar vardır. Somut ziyaretçiler, farklı düğüm türlerinin her biri için bir `visit` metodu gerçekler. Kendi arasında hiyerarşisi olan elemanları içeren bir nesne yapısı (object structure), ziyaretçinin sırayla düğümlerce kabul edilmesini yürütür; her düğüm ziyaretçinin kendi türüne ait ziyaret metodunu çağırmaktan sorumludur.

Bir kez böyle kurulduktan sonra bir düğümde yapılacak her tür işlemin düzenlendiği tek yer somut ziyaretçi sınıfları olur. Zaman içinde yeni tür işlemler, yeni bir somut ziyaretçi ya da yeni `visit` gövdeleriyle eklenebilir. Böylece geleceğe yönelik bir altyapı kurulmuş olur.

12.3 Bedel: yeni eleman türü eklemek

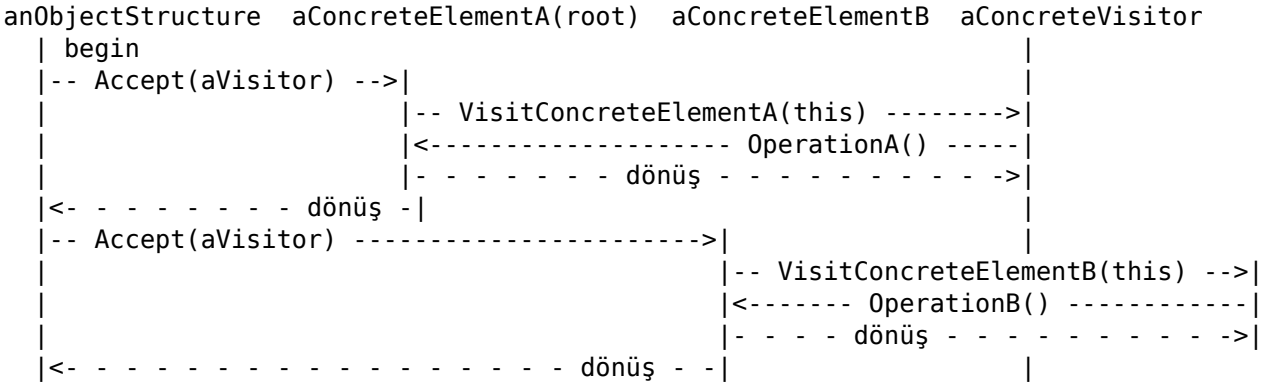
Kalıbın zayıf yönü yeni bir somut eleman türü eklemektir. Hiyerarşiye yeni bir düğüm türü eklenecekse soyut `NodeVisitor`'a o tür için yeni bir `visit` metodu eklenmelidir. Mevcut somut ziyaretçilerde bu türle ilgili yapılacak bir şey olmasa bile boş gövdeli gerçeklemeler yazılır, yeni düğümün `Accept`'i de ayrıca kodlanır. Normalde kolayca eklenebilecek bir düğüm, Visitor kalıbıyla birçok yerde değişiklik ve ekleme gerektirir. Yeni gereksinimler ve çalışma biçimleri sisteme rahatça eklenebilir, ama yeni düğüm türü eklemek zorlar; ödenen bedel budur.

> **Hocanın vurgusu:** Eleman yapısı, hiyerarşisi ve elemanların türleri değişmeyecekse Visitor kalıbı kullanılabilir. Değişecekse kullanmamak gerekir; yoksa kendi-

nize daha fazla iş çıkarmış olursunuz.

12.4 Etkileşimler

Hoca GoF'un sıra şemasını (sequence diagram) anlattı ve GoF'ta ve birçok kaynakta ihmal edilen dönüş oklarını kendisi çizdi (ekrandan, 02:10:00). Akış şöyledir:



Dolaşma işlemi nesne yapısında başlar. Yapının ilk düğümü kök düğümdür, bu yüzden `aConcreteElementA` aynı zamanda köktür. Ona `Accept` çağrılır ve ziyaretçi parametre olarak verilir. Kök, ziyaretçiyi kabul eder, ama `ConcreteElementA` türünden olduğu için ziyaretçinin `VisitConcreteElementA` metodunu, `this` ile kendisini parametre vererek çağırır. Somut ziyaretçi bu metodun içinde düğümde yapılması gereken işlemi (örnekte `OperationA()`) yürütür. İşlem tamamlanıp döner; başka işlemler varsa onlar da gidip gelir. Ziyaret bitince dönüş `Accept` çağrısına, oradan nesne yapısına kadar gider. Ardından sıradaki düğüm olan `ConcreteElementB`'ye "bu ziyaretçiyi kabul et" denir; o da "ben B türündeyim, beni B türünden ziyaret et" der. Denetim akışı tekrar nesne yapısında kalınan yere döner ve sıradaki düğümler ziyaret edilir.

Hoca, dönüş okları çizilmezse hangi metodun hangisinin içinden, hangisinin ardından çağrıldığının bu şemadan anlaşılmadığını, dönüş okları çizilince akışın netleştiğini vurguladı.

12.5 Kullanım yerleri

Slayttaki koşullar (ekrandan, 02:10:00):

- Değişen arayüzlere sahip nesnelerden oluşan bir yapıda, nesneler üzerinde farklı işlemler yapılması gerektiği anlarda VE
- bu nesne yapısındaki türlerin değişmediği ya da ender değiştiği koşullarda.

Hoca slayttaki büyük harfli "VE" bağlacının önemli bir koşul olduğunu, çünkü yeni düğüm türü eklemenin bu kalıpta epey zor olduğunu hatırlattı.

13. Visitor Uygulama Örneği: Proje Yönetimi Yazılımı

13.1 Model

Bir proje yönetimi yazılımı hazırlanıyor. Model şunları içerir:

- `Project` sınıfı hiyerarşinin kökünü oluşturur. Bir proje görevler (task) içerir.
- Bir görev, kapsamında üretilmesi gereken ara çıktılar varsa bunları `Deliverable` sınıfıyla modeller.
- `Project`, `Task` ve `Deliverable` ortak soyut `ProjectItem` sınıfından türetilmiştir; `ProjectItem`'da ad ve tanım (`name`, `description`), bunları alan kurucu ve getter/setter metotları vardır. Bu, alt sınıflarda kod tekrarını önlemek için üst sınıfta toplanmıştır.
- Görevlerin bir sorumlusu (`Contact`) olabilir, ama çözümde kullanılmaz.
- Görev, `ProjectItem` türünden bir liste tutar ve içine ara çıktılar eklenir. Hoca örnekte görevlerin alt görev içermediğini söyledi; bu yüzden şemada görevden `ProjectItem`'a parça-bütün okunu çizmediğini, çizseydi görevin alt görev içerebileceği gibi yanlış bir bilgi iletebileceğini açıkladı.

- Görevin tamamlanması için gereken süre (`timeRequired`) vardır. Ara çıktının malzeme bedeli (`materialsCost`, üretimde kullanılan malzemenin bedeli) ve üretim bedeli (`productionCost`, örneğin onu üretecek makinenin masrafı ya da işçinin maliyeti) vardır.
- `Project`'e görev eklenip çıkarılabilir, göreve ara çıktı eklenip çıkarılabilir (`add`/`remove` metotları).

Proje üzerinde raporlama hesaplamaları yapılacaktır: projenin tamamlanması için gereken süre ve projenin maliyeti. Süre için projenin bütün görevlerinin süreleri, maliyet için her görevdeki ara çıktıların bedellerinin toplamı hesaplanır. Hoca bunun bütün işlerin sırayla yapıldığı basit bir örnek olduğunu, paralel yürütülen işlerin karmaşıklığına girilmediğini söyledi.

13.2 Çözüm 1: ProjectCalculator

İlk çözümde `ProjectCalculator` sınıfı hesaplanacak projeyi alıp üye alanında tutar. Süre hesabının kodu (ekrandan, 02:30:00):

```
private Project project;
public ProjectCalculator(Project project) {
    this.project = project;
}
public double calculateTimeRequired() {
    double time = 0.0;
    for (ProjectItem item : project.getProjectItems()) {
        if (item instanceof Task) {
            time += calculateTimeRequired((Task) item);
        }
    }
    return time;
}
private double calculateTimeRequired(Task subTask) {
    double time = subTask.getTimeRequired();
    for (ProjectItem item : subTask.getProjectItems()) {
        if (item instanceof Task) {
            time += calculateTimeRequired((Task) item);
        }
    }
    // (ekranda görünen kısım burada bitiyor)
```

Kümülatif toplam değişkeni tanımlanır, projenin her elemanı için "bu bir görevse süresini hesapla" denir. Maliyet hesabında da benzer biçimde "bu bir ara çıktıysa malzeme ve üretim bedelini al, görevse onun maliyetini hesapla" denir. Hoca `instanceof` işlecine ve tip dönüşümüne (`(Task) item`) dikkat çekti. Özel yardımcı metotlar, alt görevler olsaydı onları da dolaşacak biçimde yazılmıştır; örnekte alt görev yoktur ama kolayca olabilir. Hoca hesabı public ve private iki metoda bölmenin çok gerekli olmadığını, uzun kodun anlamlı bir parçasını ileride daha kolay yönetmek için ayırdığını söyledi.

`Report` sınıfının `main` metodunda bir proje oluşturulur. Hocanın örneğinde proje bir yüksek lisans eğitimidir: iki dönemde dörder ders olmak üzere sekiz ders ve sonraki dönemde tez dersi alınır. Her dönem bir görev, o dönemde başarılması gereken dersler ara çıktı olarak modellenmiştir. Ardından bir `ProjectCalculator` oluşturulup projeye verilir ve maliyet ile süre hesaplanır.

> **Hocanın vurgusu:** Kodda "bu düğüm şu türden mi, bu türden mi" diye `instanceof` ile denetleyip tip dönüşümü yapılıyorsa, bu daha doğru bir kalıtım hiyerarşisinin ve çok biçimliliğin kullanılması gerektiğinin işaretidir. Birinci işaret switch-case'ler, öteki `instanceof`'lardır.

13.3 Çözüm 2: Visitor

İkinci çözümün şeması (ekrandan, 02:22:30), önceki çözümle aynı olan üyeleri göstermez:

```
«Abstract» ProjectItem
  - name, description : String
```

```
  + accept(ProjectVisitor) {abstract}
     ^
     |-- Deliverable   - materialsCost, productionCost : double   + accept(ProjectVisitor)
     |-- Task          - projectItems : ArrayList<ProjectItem>
     |                 - timeRequired : double                    + accept(ProjectVisitor)
     |-- Project       - projectItems : ArrayList<ProjectItem>    + accept(ProjectVisitor)
                         (Project ◆--> ProjectItem)

«interface» ProjectVisitor
  + visitDependentTask(DependentTask)   (slaytta üstü çizildi)
  + visitDeliverable(Deliverable)
  + visitTask(Task)
  + visitProject(Project)
     ^ (gerçekleme)
     |-- ProjectCostVisitor
     |-- ProjectTimeVisitor
Report (+ Report(), + main()) --> ProjectCostVisitor, ProjectTimeVisitor
```

`ProjectItem`'a soyut bir `accept` metodu eklenir; `Project`, `Task` ve `Deliverable` bu metodun gövdesini kodlar. Hoca şemadaki `visitDependentTask` metodunu ve `DependentTask` sınıfını göz ardı etmeyi istedi; örnek basit tutulmuştu, böyle bir alt sınıf kullanılmıyor. Maliyet hesabı ve süre hesabı için iki ayrı ziyaretçi vardır; önce biri, sonra öteki aynı hiyerarşide dolaştırılır.

Her elemanın `accept` metodu kendisine gelen ziyaretçiyi kabul eder ve "ben bir proje düğümüyüm, beni `visitProject` ile ziyaret et" der; `Task.accept` `visitTask`'i, `Deliverable.accept` `visitDeliverable`'ı çağırır.

`ProjectCostVisitor` kodu (ekrandan, 02:35:15):

```java
public class ProjectCostVisitor implements ProjectVisitor {
    private double cost = 0.0;

    public ProjectCostVisitor(Project project) {
        project.accept(this);
    }

    public void visitDependentTask(DependentTask p) {
        visitTask(p);
    }

    public void visitDeliverable(Deliverable p) {
        cost += p.getMaterialsCost() + p.getProductionCost();
    }

    public void visitTask(Task p) {
        for (ProjectItem item : p.getProjectItems()) {
            item.accept(this);
        }
    }
    // (ekranda görünen kısım burada bitiyor)
```

Kümülatif toplam başta sıfırdır. Nesne oluşturulur oluşturulmaz kurucu, verilen projenin dolaşımını `project.accept(this)` ile başlatır. Eski çözümde tip dönüşümü yapılan yerde artık doğrudan "bedel += malzeme bedeli + üretim bedeli" yazılır. Bir görev ziyaret edilince görevin elemanlarının her birinden ziyaretçiyi kabul etmesi istenir; proje ziyaret edilince de projenin bütün elemanlarına aynı mesaj gönderilir. Hoca bunu yuvarlandıkça büyüyen bir kar topuna benzetti. Biriken toplamı döndürmek için bir getter metodu vardır.

`ProjectTimeVisitor` kodu (ekrandan, 02:40:00):

```java
public void visitDependentTask(DependentTask p) {
    visitTask(p);
}
```

```
public void visitDeliverable(Deliverable p) {
}

public void visitTask(Task p) {
    time += p.getTimeRequired();
    for (ProjectItem item : p.getProjectItems()) {
        item.accept(this);
    }
}

public void visitProject(Project p) {
    for (ProjectItem item : p.getProjectItems()) {
        item.accept(this);
    }
}
```

Ara çıktının süreyle ilgili bir üyesi olmadığı için `visitDeliverable` gövdesi boştur. Görev ziyaret edilince önce görevin süresi eklenir, sonra alt elemanları ziyaret edilir.

`Report`'un kurucusu iki çözümde aynıdır. `main` metodunda önce bir `ProjectCostVisitor` oluşturulur ve maliyeti için getter çağrılır; sonra süre için bir `ProjectTimeVisitor` oluşturulur, oluşturulur oluşturulmaz projeyi dolaşır ve projenin kaç hafta süreceği yazdırılır.

13.4 İki çözümün karşılaştırılması

Hoca iki çözümü yan yana koydu. Eski `ProjectCalculator` süre ve maliyet için tek tek düğüm türünü `instanceof` ile denetleyip tip dönüştürüyordu. Visitor ile metot imzası hangi düğüm türünün dolaşıldığını zaten söylediği için "bu düğüm bu türden mi, öyleyse dönüştür" gibi işlemlere gerek kalmaz. Kazanç yalnız hesabın maliyet ve süre diye ikiye bölünmesi değildir: eski çözümdeki uzun metot ve yardımcı metot, yerini bir metot çağrısı ile bir döngü içinde yapılan çağrı gibi çok daha küçük ve yönetilebilir parçalara bırakır.

İki rapor çalıştırıldığında sonuçlar aynı çıktı: atlanan, eksik ya da fazla katılan bir şey olmadan her şey hesaba olması gerektiği kadar katıldı.

14. Kalıpları Öğrenmek Üzerine Soru-Cevaplar

Ders sonunda öğrenciler kalıpları öğrenmeye dair genel sorular sordu.

Soru: Bu kadar çok kalıp görüyoruz. Gerçek hayatta böyle bir senaryoya rastlamadan bunları ezberlemenin ya da aklımızda tutmaya çalışmanın bir gereği var mı?

Cevap: Kalıplar hakkında bilgi sahibi olunca, karşınıza bir problem çıktığında elinizde bir repertuar olur; gerekli bütün ayrıntıları baştan kendiniz keşfetmek zorunda kalmazsınız. Kalıpları kendi gereksinimlerinize göre belli bir düzeyde özelleştirmeniz gerekir, ama tümüyle yeniden keşfetmiş olmazsınız. Hoca bunu, yaz lastiği, kış lastiği, yüksek performans lastiği gibi çeşitlerin olduğunu bilip uygun olanı seçip kullanmaya benzetti.

Soru: Hangi kalıpların olduğunu ve temsilî olarak hangi sorunları çözdüklerini üst düzeyde bilmek yeterli olur mu, yoksa derinlemesine mi bilmek gerekir?

Cevap: Başlangıçta ilk belirtileri düzeyinde bilmek yeterlidir. Ayrıntıları üzerinde çalıştıkça uzmanlık artar. İlk başta bir kalıbın uygun olduğuna karar verirsiniz, belki sonra o kadar uygun olmadığı ortaya çıkar.

Soru: Kişisel projelerde tek başıma çalışırken kalıpları kullanmanın gerekli olup olmadığından şüpheye düşüyorum. Sektörde çalışmayan biri için bu konuyu etkili öğrenmek zor gibi geliyor.

Cevap: Kendi amaçlarınız için yazdığınız ve sonraki çalışmalarınızda altyapı sağlayacak bir yazılım da zamanla farklılaşacak, evrimleşecek ve gelişecektir. Tasarım kalıplarının en önemli avantajlarından biri değişen gereksinimlere karşı esneklik sağlayan çözümler üretmesidir; bu bağlamda kişisel projelerde de yararlıdır. Öğrenme yolculuğunun bir parçası olarak açık kaynak projelere katkı verilebilir. Oradaki geliştiriciler "bu kısım Strategy'ye göre yapılmıştır"

deyip geçer ve kalıbı bilmenizi bekler; bilmiyorsanız bir hatayı çözmek ya da bir görevi yerine getirmek için öğrenmeniz gerekir. Ayrıca tasarım kalıpları öğrenme aşamasında bile bazı şeylerin nasıl doğru yapılacağını gösterir. Kalıplara maruz kaldıkça nesneye dayalı tasarımın nasıl doğru yapılacağına dair içgörü ve tecrübe kazanılır; hoca bunu çocukların farkında olmadan gözlemle edindiği kazanımlara benzetti.

Soru: Algoritma ve veri yapılarını C ya da C++ ile öğrenmek, hazır mekanizmalar olmadığı için öğrenmeyi kolaylaştırıyor. Tasarım kalıplarını özüyle kavramak için tercih ettiğiniz bir dil var mı?

Cevap: Hoca görüşe katıldığını, ama tasarım kalıplarının adı üstünde dilden bağımsız anlatılan bir konu olduğunu söyledi. Önemli olan nesneye dayalı doğru modelleme yapmaktır. Dile bağlı çok küçük farklar vardır; örneğin Memento'da işi en sağlam biçimde nasıl kodlayacağımız dile göre değişir. Bir dil bir kalıbın anlattığı şeyi hazır bir bütün olarak içerebilir, ama bu, ayrıntılarıyla öğrenmek isteyen biri için avantaj değildir. Sonuç olarak herhangi bir dil kullanılabilir.

Örnek Sorular

Soru 1 (derste sözlü, 00:45:56): Bir TCP bağlantısının farklı durumları ve bu durumlar arasında geçişler var. Bu problem için Chain of Responsibility mi, State mi uygundur? Neden?

> **Çözüm (derste verildi):** State uygundur. Chain of Responsibility'de bir iş, onu en iyi yapacak nesneye yönlendirilir. TCP örneğinde ise tek bir iş (bağlantının kurulup sürdürülmesi) vardır ve bu işin farklı aşamaları, yani durumları vardır. Hangi durumda hangi koşulda hangi duruma geçileceği bütün kurallarıyla bellidir. Bu durum geçiş kuralları State kalıbının ayırt edici özelliğidir.

Soru 2 (derste sözlü, 02:47:37): Bir proje yönetimi yazılımı hazırlanıyor. Proje, görevler (task) ve ara çıktılar (deliverable) içerir; görevlerin süresi, ara çıktıların maliyeti vardır. (a) Hangi tasarım kalıbını seçersiniz? (b) Neden? Gerekçesiyle açıklayınız. (c) Tasarımı bir UML sınıf şemasıyla gösteriniz. (Kod yazılması istenmez.)

> **Çözüm (derste verildi):** Derste bu problem için Visitor kalıbı uygulandı (bkz. Bölüm 13). (a) Visitor. (b) Proje-görev-ara çıktı hiyerarşisinin eleman türleri değişmiyor, ama bu yapı üzerinde birden fazla işlem (süre ve maliyet hesabı) yapılması gerekiyor. İlk çözümdeki `instanceof` denetimleri ve tip dönüşümleri ortadan kalkar; her işlem kendi ziyaretçi sınıfında toplanır ve yeni bir hesap için yeni bir ziyaretçi yazmak yeter. (c) Soyut `ProjectItem` (`+ accept(ProjectVisitor) {abstract}`) ve alt sınıfları `Project`, `Task`, `Deliverable`; `«interface» ProjectVisitor` (`visitProject`, `visitTask`, `visitDeliverable`) ve gerçeklemeleri `ProjectCostVisitor`, `ProjectTimeVisitor`. Hoca şemada seçilen kalıbın ana yapısının (örneğin Strategy için bir üst sınıf ve alt sınıfları) görülebilmesini ve UML çizim kurallarına uyulmasını beklediğini söyledi.

Soru 3 (derste sözlü, 00:35:34): Kişisel finans programında vergi dışında başka hangi mali işlemler olabilir ve bunlar Strategy ile nasıl tasarlanmalıdır?

> **Çözüm (derste verildi):** Yatırım işlemleri, döviz alım satımı (sabit komisyon olan ve olmayan ülkeler), zamanla çıkan yeni ürünler (örneğin kur korumalı mevduat) ve emeklilik işlemleri. Her farklı mali işlem için Strategy kalıbı ayrıca kurulur (örneğin yatırım arayüzü ve altında `TRInvestment`, `EUInvestment`, `USInvestment`). Bütün işlemleri tek bir strateji arayüzünde toplamak da mümkündür, ama "böl ve yönet" gereği ayrı kalıplar önerilir.

Dersin Toparlaması

Bu derste davranışsal kalıplardan Strategy, State, Template Method ve Visitor işlendi. Strategy, aynı iş için birden fazla algoritmayı istemciden soyutlar; strateji dışarıdan verilir, context bütün işi ona devreder ve strateji gerektiğinde context'ten bilgi sorgular. Kişisel finans örneğinde switch-case çözümü yerini `TaxRule` arayüzüne bıraktı; strateji gerçeklemelerinin Singleton olmaya aday olduğu görüldü. Builder ile Strategy karşılaştırması, sınıf şemaları aynı olan kalıpların amaç ve denetim akışıyla ayrıldığını gösterdi.

State, nesnenin çalışma biçiminin durumuna göre tamamen değiştiği durumlar içindir ve Strategy'den durum geçiş kurallarıyla ayrılır: durumu context kendisi seçer ve geçişler sıktır. PIM örneğinde hoca kirli durumdan temize geçişin kaybolduğu bir hatayı derste düzeltti ve üst sınıfın alt sınıfları bilmemesi ilkesini vurguladı. Template Method, `final` bir şablon metotla değişmeyen adımları sabitler, değişen adımları soyut metotlarla alt sınıflara bırakır; ortak adımlar olduğu için veritabanı örneğinde Strategy yerine bu kalıp seçildi. Visitor, eleman sınıflarını değiştirmeden bir nesne yapısı üzerinde yeni işlemler tanımlamayı sağlar; `accept` ve türe özel `visit` metotlarıyla `instanceof` denetimleri ortadan kalkar, ama yeni eleman türü eklemek pahalıdır. Ders sonunda öğrenciler kalıpları öğrenmenin yararı ve dil seçimi üzerine sorular sordu.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.